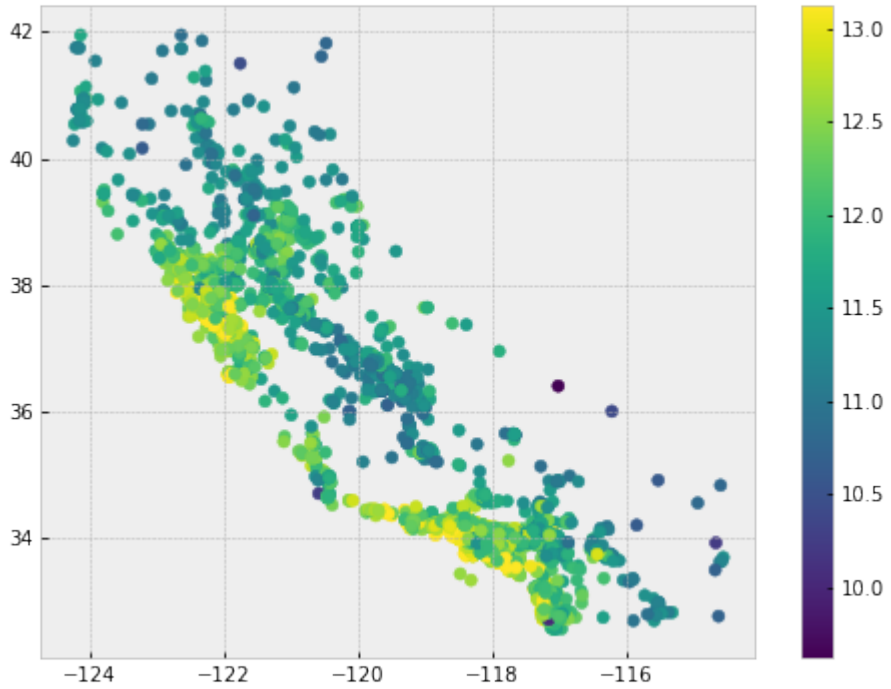


Figure A.4 The housing price data set shown as a scatter.

3. To extract our training features, we use the `torch.from_numpy()` method to convert a NumPy array to a PyTorch tensor:

```
train_x = torch.from_numpy(df.drop(["median_house_value"], axis=1).values)
```

4. We similarly do this for the log of the house prices, which are our training labels:

```
train_y = torch.from_numpy(
    df.median_house_value.values
).log().to(train_x.dtype)
```

5. We normalize the training labels `train_y` as follows:

```
train_y = (train_y - train_y.mean()) / train_y.std()
```

6. We implement the GP model as follows:

```
class GPModel(gpytorch.models.ExactGP):
    def __init__(self, train_x, train_y, likelihood):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = gpytorch.means.ConstantMean() ❶
        self.covar_module = gpytorch.kernels.ScaleKernel( ❷
            gpytorch.kernels.MaternKernel(
                nu=2.5, ❷
                ard_num_dims=train_x.shape[1] ❷
            ) ❷
        ) ❷

    def forward(self, x):
        mean_x = self.mean_module(x)
        covar_x = self.covar_module(x)
        return gpytorch.distributions.MultivariateNormal(mean_x, covar_x)
```

❶ Constant mean function.

❷ An ARD Matern 5/2 kernel with an output scale.

7. Make a likelihood whose noise is constrained to be at least 0.1 using the following code:

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>